

ALT

Capa inteligente para mejorar la accesibilidad web

Prueba de concepto: detección de imágenes sin texto alternativo, generación de sugerencias y corrección del DOM

1
Detectar

2
Sugerir

3
Aplicar

Estado actual: pipeline local funcional + integración prevista con servicio Node.js / Ollama

1. Problema que buscamos resolver

En la Web, una imagen sin atributo alt puede quedar sin una alternativa textual útil para tecnologías de asistencia. ALT explora una capa capaz de detectar ese problema en el DOM, reunir contexto, solicitar una descripción sugerida y aplicar la corrección sin exigir que el sitio original implemente manualmente toda la lógica.

- Caso inicial: elementos que no poseen el atributo alt.
- La detección se realiza sobre el DOM/HTML y se representa como un issue estructurado.
- La solución se diseñó por etapas para separar detección, contexto, sugerencia y modificación.
- El objetivo inmediato es validar el flujo técnico; no reemplazar una auditoría WCAG completa.

2. Lo construido hasta ahora

Etapas	Responsabilidad	Estado
M1	Leer HTML, parsear con JSDOM, serializar y generar salida en dist/	Completado
M2	Detectar img:not([alt]) y crear issues sin modificar el DOM	Completado
M3	Extraer contexto: título, src, caption y texto circundante	Completado
M4	Obtener suggestedAlt mediante un servicio simulado	Completado
M5	Aplicar suggestedAlt al elemento y generar HTML corregido	Completado / prueba local
Siguiente	Reemplazar el mock por un endpoint real Node.js + Ollama	En integración

3. Arquitectura actual

El procesamiento local mantiene responsabilidades separadas. La detección no decide el texto alternativo y la aplicación del cambio no necesita conocer cómo fue generado.

```
HTML
|
v
detectMissingAlt(html)
|
v
issue + context
|
v
suggestAltTexts(issues)    <- actualmente mock local
|
v
issue.suggestedAlt
|
v
applySuggestedAlt(html, issues)
|
v
HTML corregido -> dist/
```

4. Estructura del issue

```
{
  rule: "image-alt",
  type: "missing-alt",
  message: "Image is missing an alt attribute.",
  element: "img",
  html: "<img src=\"developer.jpg\">",
  context: {
    pageTitle: "Missing alternative text | Accessibility practice",
    src: "developer.jpg",
    caption: "Developer working from the office",
    surroundingText: "Developer working from the office"
  },
  suggestedAlt: "Developer working from the office"
}
```

El issue funciona como contrato entre la capa que inspecciona el DOM y la capa que propone la corrección. Esto permite reemplazar el mock sin cambiar la lógica que finalmente modifica el documento.

5. Ejemplo end-to-end

Entrada deliberadamente inaccesible

```
<figure>
  
  <figcaption>
    Developer working from the office
  </figcaption>
</figure>
```

Detección y contexto

ALT detecta que la imagen no posee alt, extrae el src y el contexto textual disponible y crea un issue de tipo missing-alt.

Sugerencia

```
suggestedAlt: "Developer working from the office"
```

Resultado generado

```
<figure>
  
  <figcaption>
    Developer working from the office
  </figcaption>
</figure>
```

El archivo original en source/ permanece sin cambios; la versión procesada se genera en dist/. En una integración sobre un navegador, el mismo concepto puede aplicarse directamente sobre el DOM vivo.

6. Arquitectura objetivo: Node.js + Ollama

El siguiente paso es sustituir la función simulada por una llamada HTTP a un servicio de backend. Ese servicio se ejecutaría en Node.js y utilizaría Ollama como capa local de inferencia para analizar la imagen y devolver una propuesta de texto alternativo.

```
ALT / navegador
|
| POST /api/suggest-alt
| issue + contexto + referencia/contenido de imagen
v
Node.js Suggested Alt API
|
| prepara solicitud multimodal
v
Ollama + modelo con capacidad de visión
|
| descripción sugerida
v
Node.js API
|
| { "suggestedAlt": "..." }
v
ALT
|
v
applySuggestedAlt()
|
v
DOM corregido
```

7. Contrato propuesto del endpoint

Ejemplo conceptual de request:

```
POST /api/suggest-alt
Content-Type: application/json

{
  "type": "missing-alt",
  "context": {
    "pageTitle": "Our engineering team",
    "src": "https://site.example/images/developer.jpg",
    "caption": null,
    "surroundingText": "Meet the people building our applications."
  }
}
```

Para que Ollama pueda describir realmente la imagen, el backend deberá recibir o poder obtener sus datos visuales (por ejemplo, bytes/base64 o una URL que el backend pueda resolver). El contexto DOM por sí solo no equivale a analizar la imagen.

Ejemplo conceptual de response:

```
{  
  "suggestedAlt": "Developer working at a laptop in an office"  
}
```

8. Separación de responsabilidades

Componente	Responsabilidad
Detector DOM	Encontrar problemas verificables, por ejemplo img sin atributo alt.
Extractor de contexto	Recolectar señales del documento sin inventar significado.
Suggested Alt API	Recibir el issue y coordinar la generación de una propuesta.
Ollama / modelo visual	Analizar la imagen junto con el contexto disponible.
Aplicador DOM	Aplicar únicamente una sugerencia válida al elemento correspondiente.

9. Controles importantes

- No sobrescribir un alt existente, incluido alt="".
- No inventar un texto genérico cuando el servicio no pueda generar una descripción fiable.
- Relacionar cada issue con el elemento correcto, incluso si varias imágenes usan el mismo src.
- Manejar timeouts, errores HTTP y respuestas inválidas sin romper el DOM.
- Limitar tamaño y tipo de imágenes enviadas al servicio y validar entradas del endpoint.
- Revisar CORS, autenticación y exposición de imágenes antes de convertir la prueba en un servicio público.
- Tratar suggestedAlt como una propuesta: la calidad debe validarse, especialmente en imágenes complejas o funcionales.

10. Próxima prueba: DOM de un sitio real

Para páginas reales, especialmente aplicaciones React/Vue/Next, conviene capturar el DOM renderizado mediante un navegador automatizado en lugar de depender únicamente del HTML inicial.

```
URL real
|
v
Playwright / navegador
|
v
DOM renderizado
|
v
detectMissingAlt()
|
v
POST Suggested Alt API
|
v
Node.js -> Ollama
|
v
suggestedAlt
|
v
inyectar alt en DOM vivo
|
v
validar resultado
```

11. Próximos hitos

- Crear el endpoint POST /api/suggest-alt en Node.js.
- Definir cómo se transportará la imagen al backend (URL resoluble o contenido de imagen).
- Integrar el backend con Ollama y un modelo con visión.
- Cambiar suggestAltTexts() para consumir el endpoint de forma asíncrona.
- Agregar manejo de errores, timeout y respuesta null.
- Probar el pipeline sobre una página de accesibilidad controlada y luego sobre un DOM real con Playwright.
- Comparar el alt sugerido con una referencia humana y medir calidad antes de automatizar correcciones a escala.

12. Visión del prototipo

ALT no se limita a detectar un error: el prototipo demuestra una arquitectura en la que una capa puede observar el DOM, construir contexto, delegar el análisis visual a un servicio especializado y aplicar una mejora accesible de forma controlada.